

一种微处理器隐藏指令分析工具的设计与实现

作者待填

2026 年 4 月

摘要

微处理器实现中可能存在规范未显式描述、工具链难以一致识别、或在不同执行环境下表现不一致的指令行为，这类差异会直接影响系统软件验证、漏洞分析与第三方审计。现有工作已经证明隐藏指令审计具有研究价值，但在多 ISA 统一工程结构、真实硬件证据链、以及新架构低成本接入方面仍存在明显缺口。本文设计并实现了一种面向 RISC 风格处理器的隐藏指令分析工具，以单条指令注入为核心，将 `memcage` 与 `ptrace` 两类执行后端、信号与反汇编联合分类、以及配置驱动的新架构后端生成纳入同一框架。当前结果表明，该工具已在 Layer A 环境下稳定完成 RISC-V 与 AArch64 的扫描与日志落盘；在 AArch64 对齐子集上，`memcage` 的 median 吞吐达到 9362.29 insn/s，高于 `ptrace` 的 8192.00 insn/s；同时，MIPS64 生成链路已完成 quick smoke，说明“配置 + 生成 + 有界人工加固”的路线具备可行性。本文当前稿的边界在于 Layer B 实机验证尚未完成，因此关于实机可复现性与 Layer A/Layer B 差异归因的结论仍保留为占位，但现有实现已经形成了一套可扩展、可复现、可审计的分析框架，并为后续板卡补证提供了直接落点。

目录

第一章 绪论	3
第二章 相关工作与技术基础	5
第三章 问题定义与设计目标	7
3.1 问题定义	7
3.2 分类口径	7
3.3 覆盖模型与边界	8
3.4 设计目标	8
第四章 系统设计	10
4.1 总体架构	10
4.2 扫描主循环	10
4.3 memcage 路径	10
4.4 ptrace 路径	11
4.5 配置驱动生成链路	11
4.6 多 worker 分片与聚合	12
第五章 实现	14
5.1 RISC-V 与 AArch64 后端	14
5.2 结果落盘与离线分析	14
5.3 MIPS 生成器案例	15
5.4 当前实现边界	15
第六章 实验与评估	16
6.1 Layer A: 稳定性与受控样例	16
6.2 Layer A: 后端吞吐对比	16
6.3 Layer B: 真实硬件复现	17
6.4 Layer A / Layer B 对照	17
6.5 Layer C: 生成器案例研究	17

目录	2
第七章 讨论与局限	20
第八章 总结与展望	21
附录 A 附录：待补实验与写作占位	24
A.1 Layer B 必补信息	24
A.2 Layer A / Layer B 对照建议	24
A.3 图表回填清单	24

第一章 绪论

处理器并不是一个天然可信的黑盒。即使厂商公开了 ISA 手册、编译器后端和常见反汇编器，处理器真实接受的编码集合、工具链对该编码的解释、以及不同执行环境中的行为结果之间仍可能存在差异。这类差异既可能体现为狭义的 undocumented instruction，也可能体现为解码器误判、模拟器工件、特权相关行为或执行期异常；对系统软件验证、漏洞分析和第三方审计而言，它们都具有直接价值。

x86 上的 `sandsifter` 工作已经证明，系统化的指令空间审计能够发现隐藏指令、硬件异常以及工具链缺陷 [4]。RISC 方向上，`iScanU` 进一步将这一路线推广到 RISC-V 与 ARMv8，并给出了 `ptrace` 与 `memcage` 两条用户态执行路径 [3]。与此同时，`armshaker` 也说明，很多看似“隐藏指令”的现象实际上需要在 CPU、解码器、模拟器与系统软件之间做细致 triage [6]。这些工作共同构成了本文的直接背景。

然而，现有方案在本文所关心的场景中仍有四个缺口。第一，多数原型以单 ISA 为主，缺少明确的多 ISA 统一工程骨架。第二，`memcage` 与 `ptrace` 往往被当作独立技巧，而不是同一系统中的互补后端。第三，容器或仿真结果与真实硬件证据之间常被分开叙述，导致可复现性边界不够清晰。第四，新架构 bring-up 仍然高度依赖手工改写后端，缺少“配置 + 生成 + 有界人工加固”的接入路径。

本文的核心思路是用统一编排框架把这些问题收束到一条主线上：前端以 Python 负责编排参数、worker 与日志聚合，中间层以共享执行核心管理“写指-执行-回收”的公共逻辑，后端以 `arch_*.c` 封装架构差异；同时保留 `memcage` 与 `ptrace` 两种执行控制，并增加配置驱动的后端生成链路，把新 ISA 的初始接入从“完全重写”收缩为“规格输入 + 可运行基线 + 明确 TODO 清单”。

围绕上述目标，本文试图回答三个问题。其一，如何在用户态环境中，对任意指令编码实现单条、可回收、可分类的执行测试。其二，如何在同一框架中组织 `memcage` 与 `ptrace`，并在多 ISA 场景下复用同一套编排逻辑。其三，当新增一种 ISA 时，能否将 bring-up 过程从完全手工重写后端收缩为配置驱动的基线生成问题。

本文的主要贡献如下。

1. 设计并实现了一套面向 RISC-V 与 AArch64 的用户态隐藏指令分析框架，统一了写指、执行、信号捕获、反汇编与分类主循环。
2. 在统一框架中组织了 `memcage` 与 `ptrace` 两类执行后端，并给出当前 AArch64 对齐子集上的 matched-subset 吞吐对比。

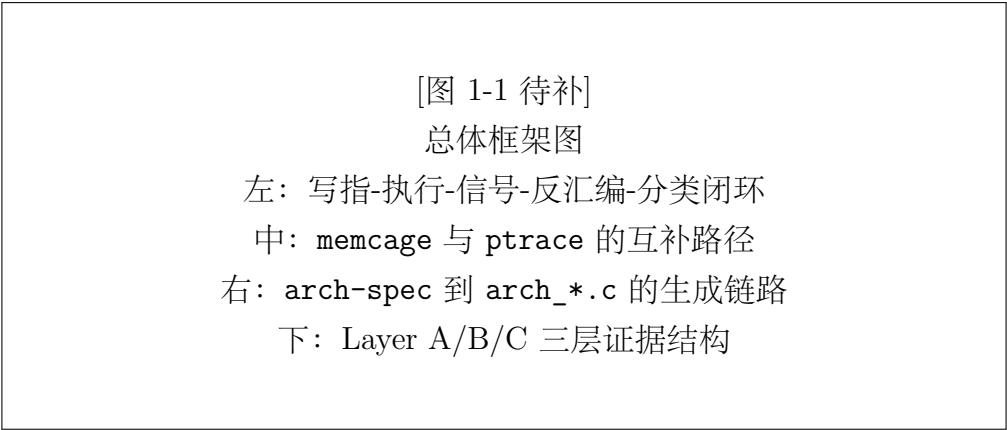


图 1.1: 本文工具的总体结构与证据路径。该图将在终稿中替换为正式框架图，用于让读者在阅读方法细节前先理解统一编排、双后端和生成器之间的关系。

- 3. 提出并实现了配置驱动的新架构生成链路，并以 MIPS64 作为首个案例验证“配置 + 生成 + 有界人工加固”的可行性。
- 4. 采用 Layer A、Layer B、Layer C 的分层证据结构；当前稿已经完成 Layer A 与 Layer C，Layer B 保留为后续实机补证部分。

当前最强结果包括三部分。第一，RISC-V 与 AArch64 的 Layer A 短程扫描已能稳定完成并落盘。第二，在 AArch64 的对齐子集上，memcage 的 median 吞吐相对 ptrace 高约 14.3%。第三，MIPS64 的生成器案例已经从 36 行规格输入产出 497 行可编译后端，并完成 quick smoke。与此同时，本文并不把当前稿写成“已经证明所有主张”的定稿：真实硬件可复现性和 Layer A/Layer B 差异归因仍待后续实验补齐。

本文余下内容组织如下。第 2 章讨论相关工作与技术背景；第 3 章给出问题定义、分类口径与设计目标；第 4 章介绍系统设计；第 5 章说明实现细节；第 6 章报告实验与评估；第 7 章讨论局限；第 8 章总结全文并给出后续工作。

第二章 相关工作与技术基础

本章按方法家族而不是按年份逐篇罗列相关工作。这样组织的目的有两点：一是让读者先看清本文试图补哪一类缺口，二是避免把“隐藏指令发现”“形式化语义”“工具链基础设施”等本质不同的问题混写成一条松散文献链。

隐藏指令发现与指令空间审计。 `sandsifter` 是这一方向最具代表性的早期工作之一，它在 x86 上通过页错误分析与搜索空间裁剪执行大规模指令审计，发现了隐藏指令、硬件异常和多类工具链 bug [4]。这项工作的意义首先在于证明了“处理器并不是可信黑盒”这一命题可以通过系统化实验方法落地验证；其次，它也为后续工作树立了一个重要边界：在变长 ISA 上，搜索空间剪枝本身就是研究问题。本文不尝试复现 x86 的复杂剪枝体系，而是将重点转向固定字长或更容易统一构造测试页的 RISC 风格 ISA，并把贡献放在执行控制、证据组织和可扩展接入上。

RISC 用户态扫描与差异 triage。 `iScanU` 是与本文最接近的先行工作。它把 `ptrace` 与 `memcage` 组织成 RISC 处理器上的用户态扫描框架，并通过信号与反汇编联合分析来识别未文档化行为 [3]。`armshaker` 则聚焦 Armv8-A，强调从用户视角观察到的“隐藏行为”未必意味着真实硬件存在 undocumented instruction，其中相当一部分现象需要在 CPU、模拟器、反汇编器与操作系统之间做细致 triage [6]。本文继承这两项工作的核心观察，但进一步把多 worker 编排、统一 raw 协议、Layer A/B/C 证据结构以及配置驱动生成链路纳入同一工程叙事。

authoritative semantics、合规测试与 formal verification。 另一条研究路线并不直接做黑盒审计，而是试图提供权威语义或在其之上构建形式化验证。Sail 系列工作给出了 ARMv8-A、RISC-V 与 CHERI-MIPS 等 ISA 的大规模可执行语义模型，这些模型已经完整到能够支持真实系统软件启动 [1]。在此基础上，`Islaris` 进一步展示了如何在 authoritative ISA semantics 之上开展机器码验证 [5]。这些工作回答的是“在已有权威语义的前提下，如何验证程序行为”，而本文回答的是“当手头只有成品 CPU、现成工具链和用户态运行环境时，如何做第三方差异审计”。因此，两者并不竞争同一问题。它们对本文最重要的启发，是迫使我们诚实界定 ground truth: `Capstone` 在本文中是 practical decoder dependency，而不是 authoritative truth。

表 2.1: 相关工作与本文的对比占位

[表 2-1 待补]

建议列：工作、目标 ISA、执行控制、ground truth 依赖、是否包含实
机证据、是否支持新架构低成本接入。

该表将依据 LITERATURE_REVIEW.md 在终稿中补齐。

执行环境与工具链基础设施。 QEMU 等模拟器和现代反汇编框架使大规模可复现实验成为可能。Bellard 的 QEMU 工作说明了跨 ISA 动态翻译和用户态模拟的工程基础 [2]。但本文并不把 QEMU 当作验证 oracle，而把它放在 Layer A 中作为受控开发与对照环境。这样的定位非常重要，因为本文在实验中已经观察到，模拟器、解码器和真实硬件之间可能形成多源差异。如果把 QEMU 或单一 decoder 直接写成“真值”，论文会陷入循环论证。基于同样原因，本文在结果解释中强调第二解码器、人工 triage 和后续实机补证，而不是把 Layer A 结果直接上升为硬件事实。

本文的位置。 综合来看，本文相对已有工作的定位可以概括为三点。第一，相比 sandsifter，本文不追求变长 ISA 的深度搜索裁剪，而强调面向 RISC 风格 ISA 的统一执行控制与扩展路线。第二，相比 iScanU，本文把“多 ISA 工程化落地”“配置驱动生成链路”和“Layer A/B/C 分层证据结构”显式纳入主要贡献。第三，相比 Sail、Islaris 等 formal 路线，本文的目标是面向现实硬件和现成工具链的黑盒审计，而不是证明实现满足形式化规范。

第三章 问题定义与设计目标

3.1 问题定义

给定目标 ISA、执行后端、待扫描编码空间和当前运行环境，本文的目标是判定每条编码在“处理器执行结果”与“反汇编器合法性判断”之间是否存在行为分歧，并输出可复现、可审计的日志与元数据。这里的“分歧”不只包括狭义的 hidden instruction，也包括特权相关拒绝、执行期异常、超时和工具链之间的不一致。

本文采用的工作假设是：在用户态可控环境中，单条指令注入、信号捕获和反汇编判断能够构成一个实用的差异审计闭环。这个闭环并不试图直接给出 authoritative semantics，而是提供一组可复核的异常候选，并把后续 triage 的上下文一并保留下来。

3.2 分类口径

当前系统使用如下工作分类。

- **H**: 执行成功，但主反汇编器不认为其为合法指令。
- **D**: 主反汇编器认为合法，但执行在测试指令处被处理器拒绝。
- **T**: 执行超时。
- **X**: 处理器进入执行路径后，在执行期间触发其他异常。
- **P**: 特权或权限相关行为。
- **UNKNOWN**: 当前实验无法稳定归类，或仍需人工 triage。

这些标签在论文叙事中已经足够表达问题空间，但在当前实现中，它们并不是完全等价的自动输出标签。尤其是 AArch64 的 controlled illegal 与 trap 路径，已经通过受控实验暴露出日志 taxonomy 的空洞。因此，本文在正文中显式区分“理论上希望表达的分类口径”和“当前程序已经稳定输出的 artifact 标签”，避免把实现边界隐藏到附录中。

表 3.1: 当前工作分类与日志自动输出能力

标签	含义	当前自动输出	当前已知缺口
H	执行成功，但主反汇编器不认为合法	部分是	RISC-V 已有 raw/filter 双口径；AArch64 当前样例尚未覆盖稳定 H
D	主反汇编器认为合法，是但执行在测试指令处被拒绝	是	AArch64 privileged-in-user 当前折叠到 D(strict)
T	执行超时	是	仅对应当前 watchdog/timeout 口径
X	执行期间触发其他异常	部分是	已知 known-disassembly exec-fault 路径未稳定进入 artifact
P	特权或权限相关行为	否	当前更多依赖人工 triage，而非独立自动标签
UNKNOWN	当前无法稳定归类	否	需结合第二解码器与人工复核产生

3.3 覆盖模型与边界

为了避免过度主张，本文在问题定义阶段就冻结三个边界。第一，扫描结果总是绑定到具体 ISA、具体执行后端、具体编码子空间和具体运行环境；脱离这些条件的泛化结论都不成立。第二，Layer A 的模拟或容器结果不能直接替代真实硬件结论。第三，用户态审计天然看不到全部微架构状态，因此它适合发现差异候选和实现偏差，但不等价于完整的 ISA 合规证明。

3.4 设计目标

本文的系统设计围绕以下目标展开。

- 1. 单条可控执行：每次只测试一个目标编码，并能在测试后回收控制流。
- 2. 程序状态可回收：局部异常不应破坏整个 harness。
- 3. 用户态可部署：尽量避免依赖内核修改或特权模块。
- 4. 多 ISA 共用编排骨架：前端逻辑不因架构差异而大规模分叉。

5. 新 ISA 接入成本可量化：明确区分可自动生成部分与必须人工加固部分。
6. 容器或仿真结果能够与后续实机结果对照。

这六个目标分别对应本文后续章节中的设计选择：第 4 章解释统一架构与双后端如何实现 G1-G4，第 5 章解释生成器如何回答 G5，第 6 章则用 Layer A/B/C 结构回应 G6。

第四章 系统设计

4.1 总体架构

本文系统由三部分组成。第一部分是 Python 编排层，负责参数解析、ISA 检测、worker 管理、结果汇总和离线分析入口。第二部分是共享执行核心 `injector_core`，负责调度当前 ISA 后端、写入测试指令、触发执行并将结果输出为统一格式。第三部分是 `arch_riscv.c`、`arch_aarch64.c` 和 `arch_mips.c` 等架构后端，用于封装指令长度、寄存器布局、黑名单规则、跳板与信号语义等 ISA 特定差异。

这样的切分有两个直接收益。其一，前端控制逻辑和后端架构差异被显式隔离，从而避免每支持一个新 ISA 都重写整条流水线。其二，日志协议可以在 C 侧执行器与 Python reader 之间保持稳定，使多 worker 并行和离线聚合都建立在统一帧格式之上。

4.2 扫描主循环

一次扫描的基本流程为：写入待测编码；布置哨兵指令；转移执行流到测试页；捕获异常信号及其上下文；调用 `Capstone` 对编码做反汇编；最后基于执行结果和反汇编结果完成分类，并将结果写入同步文件与最终日志。该主循环本身并不依赖某一特定 ISA，但它要求后端提供一组明确接口，包括测试页准备、跳板进入方式、寄存器沙箱和异常解释。

当前实现同时保留 `text` 与 `raw` 两种输出协议。`text` 输出便于人工阅读和归档，`raw` 输出面向 `sifter.py` 的 reader 线程，能够用固定长度帧传递 worker 标识、编码、反汇编状态、信号号与 `si_code` 等字段。相比完全依赖文本管道，`raw` 协议更适合多 worker 并发场景，因为它减少了解析歧义，也减轻了 reader 的字符串处理负担。

4.3 memcage 路径

`memcage` 的核心思想是在用户态构造一个受限执行环境：测试页负责注入待测指令，`trap page` 和 `guard page` 吸收大多数寄存器跳转或越界访存，超时处理逻辑负责收回死循环或等待型指令。其直接优势是吞吐更高，因为执行路径主要停留在用户态内部，不必在每次测试时都引入 `tracer/tracee` 的系统调用切换。



图 4.1: **memcage** 路径的内存布局与信号回收流程。终稿将补入真实结构图，用于解释为何该后端吞吐更高，同时也更依赖 ISA 特化 hardening。

不过，**memcage** 的高吞吐并不是“免费”的。它至少依赖三类强 ISA 特化设计。第一，哨兵指令必须既能稳定产生可识别后继行为，又不能和常见测试指令发生过强语义干扰。第二，寄存器沙箱需要把大多数通用寄存器预置到安全地址，以降低随机访存、错误返回或寄存器跳转对 harness 的破坏。第三，信号处理逻辑必须结合故障地址或等价上下文，区分“测试指令本身被拒绝”和“测试指令执行成功后命中哨兵”这两类情况。正因如此，本文把 **memcage** 明确视为高吞吐但高特化的后端。

4.4 ptrace 路径

ptrace 路径通过 tracer/tracee 模式对单条指令执行进行更强的可观察控制。与 **memcage** 相比，它更容易 bring-up，也更便于调试，因为寄存器读写、程序计数器调整和单步行为都可以通过通用接口显式观察；代价则是 stop/resume 与系统调用引入的额外开销。

从设计选择上看，本文并没有让 **ptrace** 取代 **memcage**。原因不是 **ptrace** 不够通用，而是两者回答的问题不同。**ptrace** 更适合作为默认 bring-up 路径和调试后端，而 **memcage** 更适合作为在用户态内追求更高吞吐的强化版本。也正因此，本文的生成器优先产出一个可以接入统一编排框架的基线后端，再把 **memcage** 相关 hardening 通过 TODO 清单显式暴露出来。

4.5 配置驱动生成链路

为降低新 ISA bring-up 成本，本文设计了配置驱动的后端生成器。其输入是 ISA 描述文件，例如 arch-specs/mips64el.json；输出则是可编译的基线后端 src/arch_mips.c 以及面向人工加固的 MEMCAGE_TODO 文档。这个设计有意避免宣称“零人工支持新 ISA”。相反，它明确区分：哪些信息可以安全地由配置推导，哪些部分必须依赖架构知识和后



图 4.2: `ptrace` 路径的控制流程。该图将用于强调其更强的可观察性与更高的系统调用开销。

续实测。

在当前实现中，可由配置直接描述的内容包括：`Capstone` 架构与 `mode` 宏、固定指令长度、基础黑名单、哨兵指令、寄存器组结构，以及 `ptrace` 所需的 PC/GPR 字段名。相比之下，寄存器沙箱策略、异常语义、`si_addr` 解释和特权敏感指令的黑名单收敛，则更适合通过 `TODO` 清单显式暴露给开发者。这种分工虽然保守，但更符合系统工程中的责任边界。

4.6 多 worker 分片与聚合

除执行路径外，本文的另一项关键设计是多 worker 分片。当前端以 `exhaustive` 模式并指定 `-j N` 时，编码空间会被划分为多个 `shard`，由独立 worker 分别处理。这样的设计可以在不共享复杂状态的前提下实现并发扩展，同时让每个 worker 的局部状态、最后编码和崩溃计数都能单独记录到 `run.json` 中。

与单进程内部多线程方案相比，独立进程更利于故障隔离。即使个别 worker 意外退出，前端也能够根据其分片边界和最后编码恢复局部进度，而不必整体中止全部任务。当前论文并不把 `restart` 机制包装成单独贡献，但这一工程结构为后续更大规模扫描保留了必要的稳定性空间。

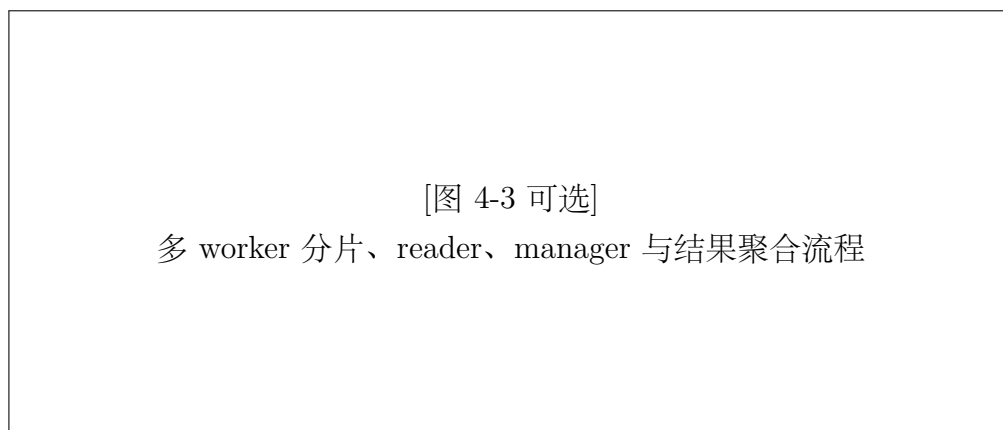


图 4.3: 多 worker 分片与结果聚合流程。该图不是主线必需，但若终稿空间允许，可用于说明前端如何在不理解 ISA 细节的前提下统一聚合多个执行器的输出。

第五章 实现

5.1 RISC-V 与 AArch64 后端

当前系统已完成 RISC-V 与 AArch64 两个主后端实现。RISC-V 路径支持压缩指令、ISA 字符串驱动的 `Capstone mode` 配置, 以及面向扩展相关假阳性的 `--filter-ext` 逻辑。AArch64 路径则采用更严格的 `mismatch` 口径, 用以减少“反汇编器可识别但用户态不可执行”的噪声样本。

RISC-V 的一个关键实现点是 `raw hidden` 与 `filtered hidden` 的双统计口径。当前代码在启用 `--filter-ext` 时同时保留 `raw_hidden_count`、过滤后 `hidden_count`、以及按扩展分组的 `filtered_hidden_ext_counts`。这样, 同一轮 Layer A 实验既能保留原始异常候选, 又能给出更接近当前 ISA 字符串视图的 `trriage` 口径, 避免为了比较 `raw H` 与 `filtered H` 而重复执行同一批编码。

AArch64 的实现选择则体现为另一种工程取舍: 当前系统更强调严格 `mismatch`, 而不是宽松 `anomaly tally`。原因在于, 若把大量用户态不可执行但可被反汇编器识别的编码一律计入“异常”, 评估章节会被噪声主导。与其追求更大的 `anomaly` 计数, 本文更希望保留一个可解释的对照基线。

5.2 结果落盘与离线分析

运行过程中, 系统通过 `sync` 文件提供实时观察入口, 并在收尾阶段生成 `data/log` 与 `run.json`。其中, `data/log` 更适合作为实验章节中的主结果来源, 因为它直接体现测试条数、分类计数和代表性 `artifact`; `run.json` 则更适合记录命令行、`worker` 分片、运行时长、过滤选项和局部统计等元数据。

为了支撑后续人工 `trriage`, 当前实现还冻结了 `artifact bundle schema` 和 `second-decoder wrapper` 相关约定。这样做的目的不是把所有复杂性都提前自动化, 而是保证在发现有争议的样本后, 研究者可以追溯到足够完整的上下文, 而不是只剩下一个最终计数。

离线分析侧主要由 `summarize.py` 与 `analysis/` 辅助脚本承担。这些工具负责对最终日志做家族聚合、代表样例展示和 CSV 导出。对一篇强调“可审计”和“可扩展”的论文来说, 这部分内容并不是附属脚本, 而是实验解释链路的一部分。

5.3 MIPS 生成器案例

MIPS64 是本文当前的首个生成器案例。配置文件 `arch-specs/mips64el.json` 仅 36 行，却能够驱动生成器产出 497 行的 `src/arch_mips.c` 基线实现，并自动生成包含 3 条人工检查项的 `src/arch_mips.c.MEMCAGE_TODO.md`。这说明生成器已经能够回答一个现实问题：当研究者第一次引入新 ISA 时，至少可以快速获得一个可编译、可接入统一编排框架的基线后端。

更重要的是，MIPS 案例把自动化边界写得足够具体。自动化已经能生成基础哨兵、黑名单、`ptrace` 接口骨架和 `Capstone` 配置，但寄存器沙箱语义、`BREAK` 哨兵行为以及 `cache/TLB` 敏感指令的黑名单收敛仍必须依赖人工加固。也就是说，本文对“可扩展性”的回答不是“自动替代 ISA 工程”，而是“把高重复劳动压缩到规格输入和基线生成，把剩余人工工作暴露得更清楚”。

5.4 当前实现边界

本文在实现章明确写出三条边界。第一，当前自动分类输出尚未稳定覆盖全部 `H/D/P/T/X/UNKNOWN` 路径，特别是 `illegal-at-test` 与 `known-disassembly exec-fault` 样例仍存在日志空洞。第二，当前所有吞吐结果都来自 Layer A，而非真实硬件。第三，生成器产出的 MIPS 后端虽然已经可运行，但其成熟度还不能与现有 `RISC-V`、`AArch64` 后端等量齐观。

把这些边界写在实现章而不是只放在结论里，有助于让后续实验结果与实现状态保持一致，也能降低读者对当前稿“已经完整闭环”的误读。

第六章 实验与评估

本章对应当前稿中已经收紧后的 claim 集合。具体来说，§6.1 对应主循环稳定性与部分 controlled path，§6.2 对应 AArch64 对齐子集上的吞吐对比，§6.3 与 §6.4 保留给后续实机补证，§6.5 对应 MIPS 生成器案例。这样的组织方式直接继承了 Layer A/B/C 证据结构，避免把当前还没有数据支撑的主张写得过早。

6.1 Layer A：稳定性与受控样例

本节的核心问题不是“当前是否已经发现新的硬件隐藏指令”，而是“现有框架在受控 Layer A 环境中是否稳定、可审计，并能否通过小规模样例验证分类链路的关键路径”。在这一口径下，当前实验已经足以支撑主循环稳定性和部分 controlled path 的论述。

R001 至 R003 表明，当前框架在声明的 Linux 容器环境下已经能够稳定完成短程扫描并产出完整日志。R004 和 R006 进一步说明，合法样例与 privileged-in-user 样例都能走通当前链路。与此同时，R005 与 R007 也提供了一个同样重要的负结果：illegal-at-test 与 known-disassembly exec-fault 这两类路径虽然已经在受控实验中暴露出来，但当前分类器尚未把它们稳定输出为 artifact。

因此，本文在当前稿中采用更收缩的表达：系统已经能够稳定表达 legal 与 privileged-in-user 两类受控路径，并通过 illegal/trap 样例暴露 taxonomy/logging gap。这个结论比“分类器已完整覆盖 H/D/P/T/X”更弱，但与现有证据是一致的。

6.2 Layer A：后端吞吐对比

本节只测试一个收缩后的 claim：在当前 AArch64 对齐子集与当前 Layer A 环境中，memcage 的 median 吞吐高于 ptrace。我们不把这个结论外推为“memcage 一般优于 ptrace”，因为当前数据只覆盖一个架构、一个对齐窗口和三次重复实验。

这组实验有两个值得保留的点。第一，两组实验在 coverage 上保持一致，每次都测试 262144 条编码，并记录到相同数量的 D(strict)；这使得对比建立在相同工作量上，而不是建立在不同异常分布之上。第二，在这一严格对齐前提下，memcage 的 median 吞吐约高 14.3%。因此，本文采用的最稳妥表述是：在当前 matched subset 上，memcage

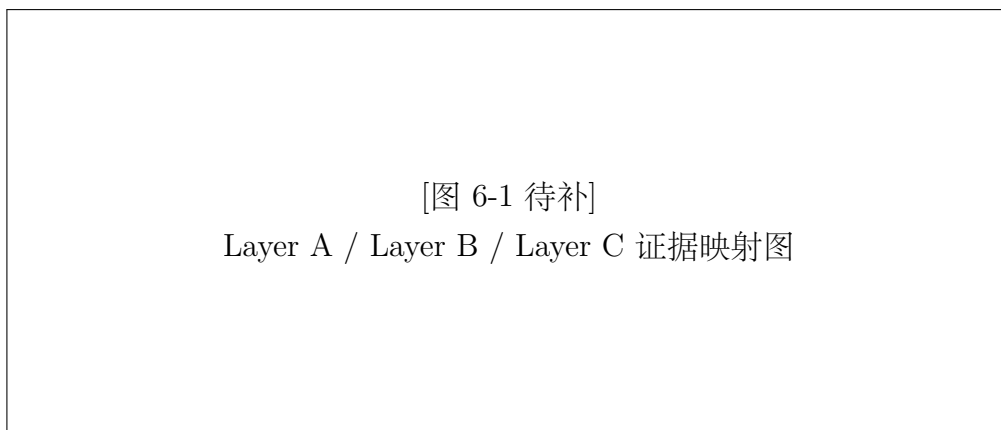


图 6.1: 本文评估的三层结构。Layer A 用于开发、sanity 与 matched-subset 对照；Layer B 用于真实硬件补证；Layer C 用于量化新 ISA 接入成本。

表现出稳定但有限的吞吐优势。

6.3 Layer B: 真实硬件复现

本节保留给后续 RISC-V 板卡实验补证。终稿至少应补入以下信息：板卡型号、SoC 或 stepping、Linux 内核版本、Capstone 版本、实验命令、三次复跑结果以及日志归档目录。该部分完成后，本文才能就真实硬件可复现性做出正面陈述。

6.4 Layer A / Layer B 对照

本节同样保留为占位。正式补写时，应预先固定对齐子集，例如相同的 `-b/-e` 范围或相同 seed，并把差异显式分桶为模拟器差异、解码器差异、特权或模式差异、以及状态依赖差异。核心目标不是追求“完全一致”，而是使观察到的差异可以解释。

6.5 Layer C: 生成器案例研究

Layer C 的问题不是“生成器能否完全替代人工实现”，而是“它是否已经把新 ISA bring-up 从零开始编码，转化为规格输入驱动的可运行基线生成问题”。围绕这个问题，当前 MIPS64 quick smoke 已经给出足够直接的证据。

这组结果说明，生成器已经能够显著降低新 ISA 接入的初始成本，但它还没有把全部工程工作自动化。更准确的说法是：当前生成链路把新架构接入问题压缩成一个小规格输入、一个可运行基线后端，以及一个显式人工检查清单。这种输出形态本身就是一种可复用的工程结论。

表 6.1: Layer A 稳定性与受控样例实验汇总

Run ID	平台/后端	子集/样例	主要结果
R001	RISC-V memcage	0x00000000--0x000000ff	Tested=4064, Raw H=31, H=0, D=97, T=33, X=32
R002	RISC-V memcage	0x00010000--0x00010fff	Tested=992, worker crash count 为 0
R003	AArch64 ptrace	0x00000000--0x00000fff	Tested=65536, D(strict)=65536
R004	AArch64 ptrace	legal case	无异常 artifact
R005	AArch64 ptrace	illegal case	运行结束但无 artifact, 暴露 taxonomy/logging gap
R006	AArch64 ptrace	privileged case	D(strict)=1
R007	AArch64 ptrace	trap candidates	运行结束但无 artifact, 暴露 exec-fault/trap logging gap

表 6.2: AArch64 matched subset 上的后端吞吐对比

后端	重复次数	扫描范围	Tested	median 吞吐	备注
memcage	3	0x00000000--0x00000fff	362144	9362.29 insn/s	三次 约为 8456.26、9362.29、9362.29
ptrace	3	同上	262144	8192.00 insn/s	三次 约为 7489.83、8192.00、8456.26

表 6.3: Layer B 实机环境与复现实验占位

[表 6-4 待补]
建议列：板卡、stepping/SoC、内核、命令、三次复跑结果、日志目录。

表 6.4: Layer A / Layer B 差异分桶占位

[表 6-5 待补]
建议列：子集定义、Layer A 结果、Layer B 结果、差异类型、解释。

表 6.5: MIPS64 生成器案例的最小量化信息

项目	当前值	含义
配置文件行数	36	MIPS64 规格输入规模
生成后端行数	497	自动生成的 <code>src/arch_mips.c</code>
人工检查项	3	<code>MEMCAGE_TODO</code> 中尚未关闭的条目
quick smoke 时间	16.27s	从调用脚本到得到可运行结果的墙 钟时间

第七章 讨论与局限

本文当前稿的第一个局限是 ground truth 边界。虽然 Capstone 为大规模指令扫描提供了现实可用的 ISA 视图，但它并不等价于 authoritative semantics。因此，凡是 decoder-related 的结论，都必须结合第二解码器、人工 triage 与后续实机验证理解。也正因如此，本文在 Layer A 结果解释中始终把当前输出视为 triage 证据，而不是直接上升为硬件事实。

第二个局限是分类输出尚未闭环。受控实验已经证明，现有实现能够稳定表达部分路径，但 illegal-at-test 与 known-disassembly exec-fault 这两类样例还没有稳定进入 artifact 日志。这个缺口既是当前实现的边界，也为后续工作指明了直接方向：在不破坏现有日志结构的前提下，补齐分类器对这些路径的落盘能力。

第三个局限是用户态方法的天然视野限制。用户态扫描非常适合做大范围差异审计，但它天然看不到全部处理器状态空间，尤其难以覆盖深层特权状态、缓存与 TLB 敏感路径，以及强状态依赖行为。因此，“未发现新隐藏指令”不应被解释为“处理器完全正确”；它最多说明，在当前子空间、当前环境和当前口径下，没有观察到足以支持该主张的证据。

第四个局限是 Layer B 尚未完成。当前论文已经可以围绕 Layer A 与 Layer C 形成一版结构完整的中文初稿，但只要没有真实硬件结果，任何关于实机可复现性和 Layer A/Layer B 差异归因的结论都必须保留为占位。这一点不仅影响实验章，也影响引言和结论中的论证强度。

最后，生成器的边界同样需要写清。本文并不声称配置生成器可以自动替代 ISA 工程，而是声称它能够把 bring-up 的高重复劳动前移到规格描述和基线生成阶段。对 memcage 这样强依赖 ISA 语义的高性能后端来说，后续人工 hardening 仍然不可避免。

第八章 总结与展望

本文围绕“如何在用户态环境中对多 ISA 处理器做可扩展、可复现、可审计的隐藏指令差异分析”这一问题，设计并实现了一套统一框架。该框架以单条可控执行为核心，用共享执行核心和架构后端分离公共逻辑与 ISA 差异，在同一系统中组织了 `memcage` 与 `ptrace` 两类互补后端，并增加了面向新 ISA 的配置驱动生成链路。

在当前已经完成的工作中，Layer A 结果支持主循环稳定性、部分 controlled path 和 AArch64 对齐子集上的吞吐对比；Layer C 结果支持 MIPS64 生成器案例的可运行性与 bring-up 成本量化。与此同时，本文也明确保留了当前证据尚不支持的主张：真实硬件可复现性、Layer A/Layer B 差异归因，以及完整闭环的分类输出能力。

后续工作将沿三个方向继续推进。第一，完成至少一台 RISC-V 板卡上的 bounded campaign，补齐 Layer B 的直接证据。第二，围绕 illegal-at-test 和 known-disassembly exec-fault 两类路径补齐日志 taxonomy。第三，把当前生成器推广到更多 ISA，并探索与 authoritative semantics 或合规测试框架的结合方式。完成这些工作后，本文将能够从“Layer A + Layer C 为主体的初稿”自然过渡到更完整的学位论文定稿。

参考文献

- [1] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–31, January 2019. ISSN 2475-1421. doi: 10.1145/3290384. URL <http://dx.doi.org/10.1145/3290384>.
- [2] Fabrice Bellard. QEMU, a fast and portable dynamic translator. In *2005 USENIX Annual Technical Conference (USENIX ATC 05)*, Anaheim, CA, April 2005. USENIX Association. URL <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>.
- [3] Rens Dofferhoff, Michael Goebel, Kristian Rietveld, and Erik van der Kouwe. iscanu: A portable scanner for undocumented instructions on RISC processors. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 306–317. IEEE, June 2020. doi: 10.1109/dsn48063.2020.00047. URL <http://dx.doi.org/10.1109/DSN48063.2020.00047>.
- [4] Christopher Domas. Breaking the x86 isa. Black Hat USA 2017 whitepaper, 2017. URL <https://blackhat.com/docs/us-17/thursday/us-17-Domas-Breaking-The-x86-Instruction-Set-wp.pdf>. Accessed 2026-04-15.
- [5] Michael Sammler, Angus Hammond, Rodolphe Lepigre, Brian Campbell, Jean Pichon-Pharabod, Derek Dreyer, Deepak Garg, and Peter Sewell. Islaris: verification of machine code against authoritative ISA semantics. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI '22*, pages 825–840. ACM, June 2022. doi: 10.1145/3519939.3523434. URL <http://dx.doi.org/10.1145/3519939.3523434>.
- [6] Fredrik Strupe and Rakesh Kumar. Uncovering hidden instructions in Armv8-A

implementations. In *Hardware and Architectural Support for Security and Privacy*, HASP '20, pages 1–9. ACM, October 2020. doi: 10.1145/3458903.3458906. URL <http://dx.doi.org/10.1145/3458903.3458906>.

附录 A 附录：待补实验与写作占位

A.1 Layer B 必补信息

- 板卡型号、SoC 或 stepping
- Linux 内核版本与 `uname -a`
- Capstone 版本与提交哈希
- 实验命令、运行时间与日志目录
- 至少三次复跑结果

A.2 Layer A / Layer B 对照建议

- 固定扫描范围或固定 seed
- 明确启用的扩展集合与过滤选项
- 将差异分桶为模拟器、解码器、特权或状态依赖差异
- 对代表性样例保留 second-decoder 与人工 triage 记录

A.3 图表回填清单

当前 paper/ 目录中仍保留以下正式占位。

- 图 1-1: 总体框架图
- 图 4-1: memcage 内存布局与信号路径
- 图 4-2: ptrace workflow
- 图 6-1: Layer A/B/C 证据映射图
- 表 2-1: 相关工作对比表

- 表 6-4: Layer B 实机复现实验表
- 表 6-5: Layer A / Layer B 差异分桶表